Pneumonia Detection from Chest X-Rays using CNN and Grad-CAM

A Project Report

submitted in partial fulfilment of the requirements for the award of the degree of

Master of Science (Data Science)

by

Piush Kole

2548421

Under the guidance of

Dr. Deepa S

Department of Computer Science

CHRIST (Deemed to be University), Bangalore Yeshwanthpur Campus

AUGUST 2026

ABSTRACT

Pneumonia is a serious lung infection that is commonly diagnosed from chest X-ray images. This project builds a small Convolutional Neural Network (CNN) that classifies chest X-rays as Normal or Pneumonia, and adds an explainability layer using Grad-CAM (Gradient-weighted Class Activation Mapping) implemented from scratch. The publicly available PneumoniaMNIST dataset from the MedMNIST collection is used; it contains 4,708 training, 524 validation and 624 test grayscale images and is downloaded automatically through the medmnist API. The CNN has three convolutional blocks followed by a dense classifier and is trained for ten epochs with the Adam optimizer and binary cross-entropy loss. The model is evaluated using accuracy and loss curves, a classification report and a confusion matrix on the unseen test set, where it reaches a test accuracy of about 0.7949 and correctly detects every pneumonia case. Grad-CAM heatmaps are generated for individual predictions and often highlight lung-field regions, giving a simple visual check on the model's reasoning. The project delivers a complete, reproducible and interpretable deep learning workflow for medical image classification.

1. INTRODUCTION

Chest X-ray is the most common radiological test used to detect pneumonia, an infection that causes the air sacs of the lungs to fill with fluid and appear as cloudy white areas on the image. Reading these X-rays requires a trained radiologist, and manual reading is slow and can vary between observers. Deep learning models can help by screening images automatically, but a plain classifier only outputs a label and gives no reason for its decision, which makes it hard to trust in a medical setting.

This project has two goals: (1) train a CNN that classifies chest X-rays as Normal or Pneumonia with good accuracy, and (2) use Grad-CAM to produce a heatmap for every prediction that shows which part of the X-ray the model looked at. Grad-CAM is implemented manually with TensorFlow, without any external explainability library.

1.1 Objectives

- Load the PneumoniaMNIST dataset with its standard train/validation/test splits using the medmnist API.
- Pre-process the images (normalisation and reshaping).
- Build and train a compact three-block CNN for binary classification.
- Evaluate the model with learning curves, a classification report and a confusion matrix.
- Implement Grad-CAM from scratch and visualise heatmaps on test images.

2. DATASET

PneumoniaMNIST is part of the MedMNIST v2 collection and is derived from a large public paediatric chest X-ray dataset. Each image is a 28x28 grayscale radiograph and each label is binary (0 = Normal, 1 = Pneumonia). The medmnist library downloads the dataset automatically the first time it runs and caches it locally, so no account or API key is needed.

Split	**Number of Images**
Train	4,708
Validation	524
Test	624
Total	5,856

Figure 1 shows a sample of training images with their labels. Inspecting the data before modelling confirms the images load correctly and shows the visual difference between the two classes.

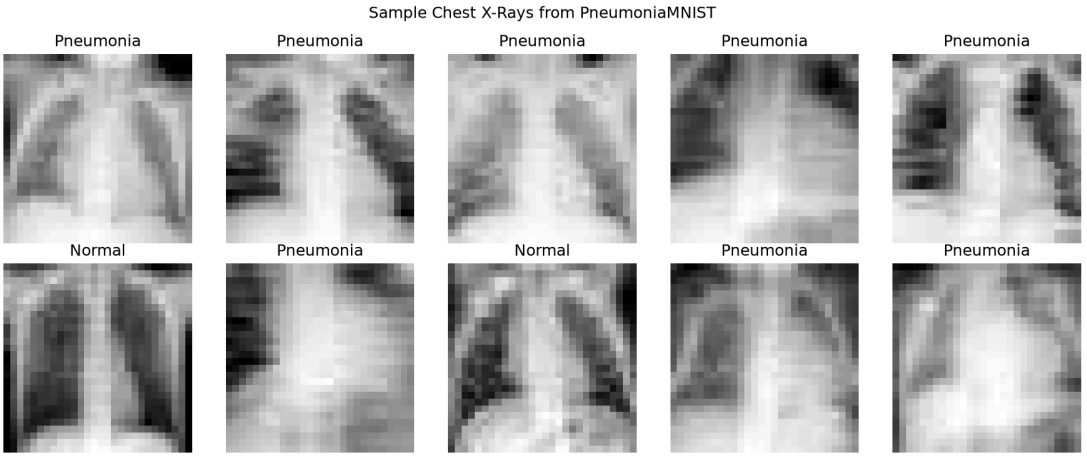


Figure 1: Sample chest X-rays from PneumoniaMNIST

3. METHODOLOGY

3.1 Pre-processing

Pixel values are scaled from the range 0-255 to 0-1 to help the network train stably, a channel axis is added so each image has shape (28, 28, 1), and the labels are flattened to a 1-D integer array.

3.2 CNN Architecture

The model is built with the Keras Functional API and has three convolutional blocks. Each block is Conv2D + ReLU + MaxPooling, with the number of filters increasing from

16 to 32 to 64. The final convolutional layer is named last_conv because Grad-CAM needs to read its output. The classifier head flattens the feature maps and applies a 64-unit dense layer, dropout of 0.3 and a single sigmoid output that gives the probability of pneumonia.

Layer	**Output Shape**
Input	28 x 28 x 1
Conv2D (16, 3x3, ReLU) + MaxPool	14 x 14 x 16
Conv2D (32, 3x3, ReLU) + MaxPool	7 x 7 x 32
Conv2D (64, 3x3, ReLU) 'last_conv' + MaxPool	3 x 3 x 64
Flatten + Dense(64) + Dropout(0.3)	64
Dense(1, sigmoid)	1

The model is compiled with the Adam optimizer and binary cross-entropy loss, and trained for 10 epochs with a batch size of 32.

3.3 Grad-CAM

Grad-CAM answers the question: which regions of the input image most influenced the prediction? The steps are: (1) run the image through the network and record the feature maps of last_conv and the output score; (2) compute the gradient of the score with respect to those feature maps using tf.GradientTape; (3) average each gradient map to get one weight per channel; (4) take the weighted sum of the feature maps and apply ReLU; (5) normalise to 0-1, resize to the image size and overlay it on the X-ray as a colour heatmap.

4. IMPLEMENTATION

The project is a single Jupyter notebook written in Python 3. Random seeds are fixed (numpy and TensorFlow = 42) so the results are reproducible. The libraries used are:

- tensorflow / keras - build and train the CNN
- medmnist - download and load the PneumoniaMNIST dataset
- numpy - array maths and normalisation
- matplotlib - all plots and figures
- opencv-python - resize and blend the Grad-CAM heatmap
- scikit-learn - classification report and confusion matrix

The model has 60,289 trainable parameters. The two key code fragments are shown below.

```
def build_model(input_shape):
    inputs = keras.Input(shape=input_shape)
    x = layers.Conv2D(16, 3, activation='relu', padding='same', name='conv1')(inputs)
    x = layers.MaxPooling2D(2)(x)
```

```
    x = layers.Conv2D(32, 3, activation='relu', padding='same', name='conv2')(x)
    x = layers.MaxPooling2D(2)(x)
    x = layers.Conv2D(64, 3, activation='relu', padding='same', name='last_conv')(x)
    x = layers.MaxPooling2D(2)(x)
    x = layers.Flatten()(x)
    x = layers.Dense(64, activation='relu')(x)
    x = layers.Dropout(0.3)(x)
    outputs = layers.Dense(1, activation='sigmoid')(x)
    return keras.Model(inputs, outputs)


def compute_gradcam_heatmap(model, img_array, last_conv_layer_name='last_conv'):
    grad_model = keras.models.Model(
        model.inputs,
        [model.get_layer(last_conv_layer_name).output, model.output])
    with tf.GradientTape() as tape:
        conv_output, predictions = grad_model(img_array)
        class_score = predictions[:, 0]
    grads = tape.gradient(class_score, conv_output)
    pooled_grads = tf.reduce_mean(grads, axis=(0, 1, 2))
    heatmap = conv_output[0] @ pooled_grads[..., tf.newaxis]
    heatmap = tf.squeeze(heatmap)
    heatmap = tf.maximum(heatmap, 0) / tf.math.reduce_max(heatmap)
    return heatmap.numpy()
```

5. RESULTS AND DISCUSSION

The model was trained for 10 epochs. The per-epoch training log is shown below.

Epoch	Train loss	Train acc.	Val loss	Val acc.
1	0.3821	0.8299	0.1800	0.9408
2	0.1778	0.9284	0.1238	0.9580
3	0.1454	0.9460	0.1043	0.9637
4	0.1261	0.9511	0.0989	0.9656
5	0.1163	0.9573	0.0915	0.9695
6	0.1020	0.9618	0.0977	0.9599
7	0.0953	0.9643	0.0992	0.9618
8	0.0876	0.9673	0.0955	0.9656
9	0.0802	0.9696	0.0922	0.9695
10	0.0764	0.9718	0.1260	0.9618

Figure 2 shows the same information as curves. Training accuracy rises steadily to about 0.97 and the validation accuracy stays around 0.96, which means the model is learning and is not badly over-fitting during training.

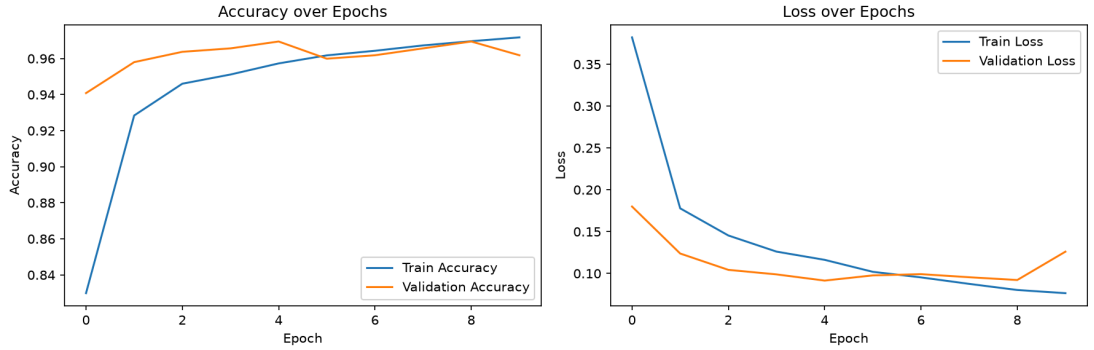


Figure 2: Training and validation accuracy and loss

On the 624-image test set the model achieved:

Metric	Value
Test accuracy	0.7949
Test loss	1.0074
Final training accuracy	0.9718
Final validation accuracy	0.9618

The full classification report for the test set is given below.

Class	Precision	Recall	F1-score	Support
Normal	1.00	0.45	0.62	234
Pneumonia	0.75	1.00	0.86	390
Accuracy			0.79	624
Macro Avg	0.88	0.73	0.74	624
Weighted Avg	0.85	0.79	0.77	624

Figure 3 is the confusion matrix. The model correctly identified all 390 pneumonia cases (recall = 1.00, zero false negatives), but it also labelled 128 of the 234 normal X-rays as pneumonia. This behaviour is caused by the class imbalance in the training data (1,214 Normal vs 3,494 Pneumonia images) and by the known difference between the PneumoniaMNIST validation and test distributions, which is why validation accuracy (about 0.96) is higher than test accuracy (about 0.7949). For a screening tool this trade-off is acceptable, because missing a pneumonia case (a false negative) is more dangerous than a false alarm.

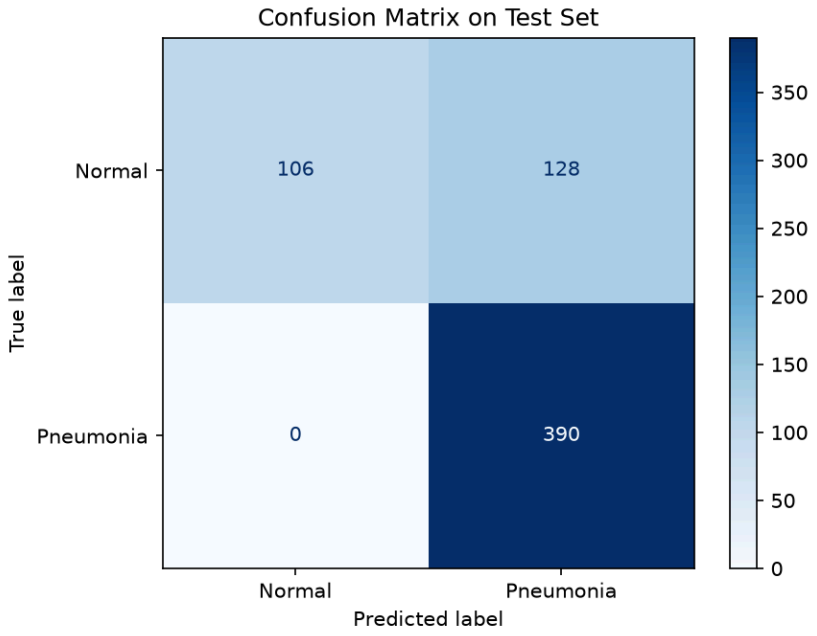


Figure 3: Confusion matrix on the test set

Figure 4 shows Grad-CAM results for five test images: the original X-ray, the heatmap, and the heatmap overlaid on the X-ray with the true and predicted labels. The heatmaps are coarse because the last convolutional layer is only 7x7 before it is upsampled, but for most pneumonia cases the warm (red) region falls on a lung area rather than on the image border or corner markers, which is a useful sanity check on the model's reasoning.

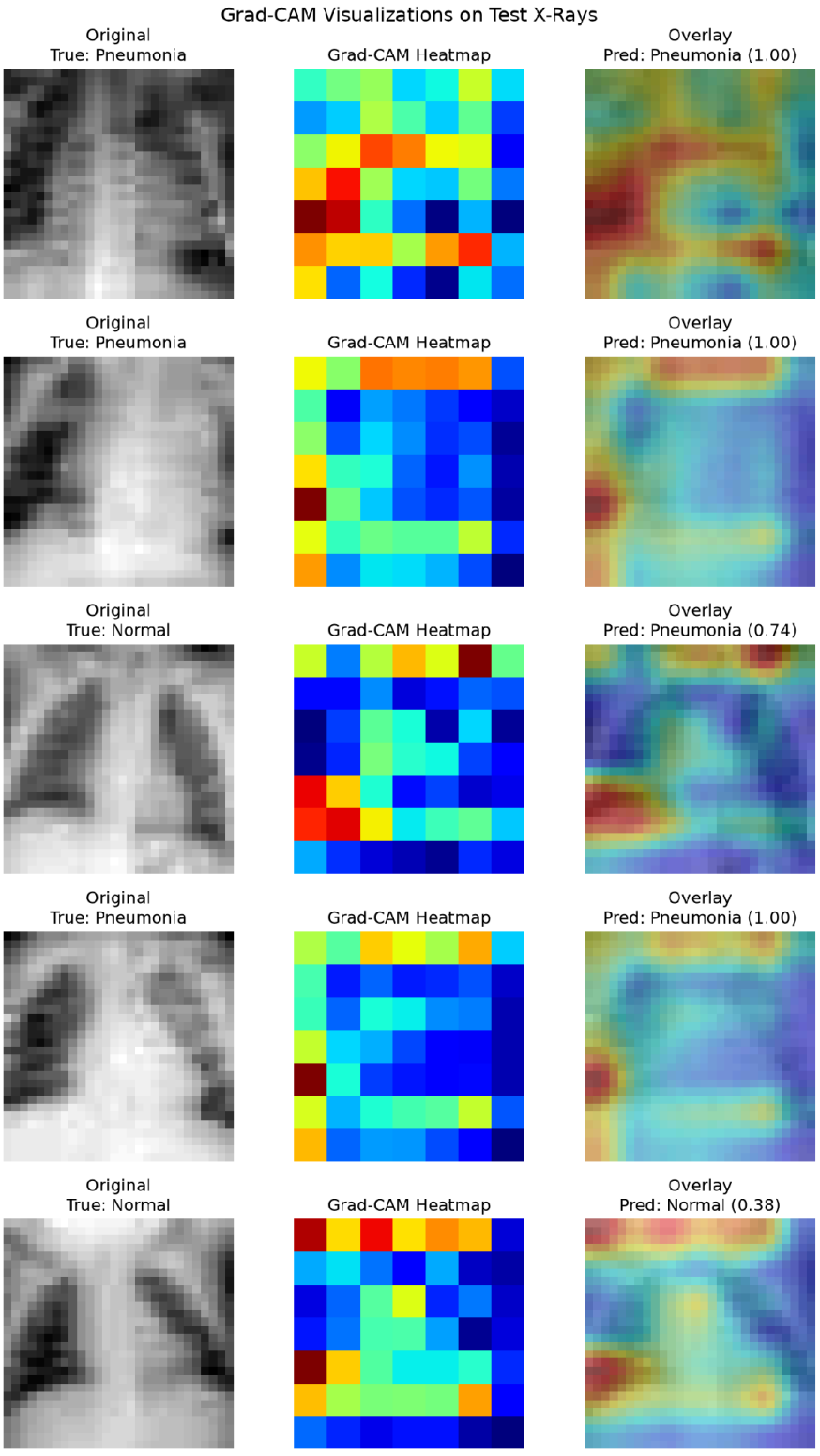


Figure 4: Grad-CAM heatmaps and overlays on five test images

6. CONCLUSION

This project built a compact CNN that classifies chest X-rays as Normal or Pneumonia and reached a test accuracy of about 0.7949 on the PneumoniaMNIST dataset, with a

recall of 1.00 for the pneumonia class. Grad-CAM was implemented from scratch with tf.GradientTape and used to produce a heatmap for every prediction. The workflow is fully reproducible (fixed random seeds) and needs no manual dataset download.

Limitations: the model over-predicts pneumonia because of class imbalance and a train/test distribution shift, the images are small (28x28), the dataset comes from a single source, and the system is for education only and must not be used for real diagnosis. Future work: handle class imbalance with class weights or oversampling, use transfer learning with a pre-trained backbone, use higher-resolution images and data augmentation, and try sharper explanation methods such as Grad-CAM++.

REFERENCES

[1] Yang, Jiancheng, et al. "MedMNIST v2 - A Large-Scale Lightweight Benchmark for 2D and 3D Biomedical Image Classification." Scientific Data 10, 2023: 41.

[2] Kermany, Daniel S., et al. "Identifying Medical Diagnoses and Treatable Diseases by Image-Based Deep Learning." Cell 172.5, 1122-1131. 2018.

[3] Selvaraju, Ramprasaath R., et al. "Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization." IEEE ICCV. 2017: 618-626.

[4] Abadi, Martin, et al. "TensorFlow: Large-Scale Machine Learning on Heterogeneous Distributed Systems." arXiv:1603.04467. 2016.

[5] Pedregosa, Fabian, et al. "Scikit-learn: Machine Learning in Python." Journal of Machine Learning Research 12, 2825-2830. 2011.

[6] MedMNIST Project. "MedMNIST." 2023. Accessed 28 Aug. 2026 <https://medmnist.com/>.